Thread Pool

Season: 5 · Concurrency · Category: Concurrency · Level: Intermediate

Intent

- Reuse workers instead of spawning unbounded threads.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Taxi stand with a fixed fleet.

When to Use

- Concurrent / parallel workloads.
- The design force named in Intent shows up more than once.

When Not to Use

- Pure CPU-bound work already parallelized.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.concurrency.threadpool;

import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.atomic.AtomicBoolean;

/** Educational pool — prefer Executors / virtual threads in real code. */
public final class SimpleThreadPool implements AutoCloseable {
    private final BlockingQueue<Runnable> queue = new LinkedBlockingQueue<>();
    private final Thread[] workers;
    private final AtomicBoolean open = new AtomicBoolean(true);

    public SimpleThreadPool(int size) {
        workers = new Thread[size];
        for (int i = 0; i < size; i++) {
            workers[i] = Thread.ofVirtual().name("worker-" + i).start(() -> {
                while (open.get() || !queue.isEmpty()) {
                    try {
                        var task = queue.poll(50, java.util.concurrent.TimeUnit.MILLISECONDS);
                        if (task != null) task.run();
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
            });
        }
    }
```

Classic Implementation (2)

```
    public void submit(Runnable task) {
        if (!open.get()) throw new IllegalStateException("closed");
        queue.add(task);
    }

    @Override
    public void close() {
        open.set(false);
        for (var w : workers) {
            try { w.join(1000); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }
    }
}
```

Modern Java 25 / Functional

- Executors.newVirtualThreadPerTaskExecutor() is the modern default for blocking tasks.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: concurrency/threadpool/
```

Pitfalls

- Prefer java.util.concurrent and virtual threads over custom pools.
- Always interrupt/shutdown cleanly.

Interview Q&A

- Q: Virtual threads vs platform threads? A: Virtual threads excel at blocking I/O concurrency; still bound by CPU for pure compute.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Reuse workers instead of spawning unbounded threads.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Taxi stand with a fixed fleet. [0:40] Intent — State the design force in one sentence: Reuse workers...